Answering

Answer

document

nswer questions about
we take a question
passages in the
. An example app
e.
to the pipeline
nt.
English examples
ng said that,
model for the
model.

Smaller

example, which
nt address.

a first step is to chunk the text into smaller pieces so
n embed each piece later. Here we can use LangChain's
ilt-in split_documents method from the
RecursiveCharacterTextSplitter module. We can define the
hunk size in characters (we choose 300) & if we want
e chunks to overlap by a certain no. of characters
(we choose 0).

```
wget 'https://docs.google.com/uc?export=download&id=
1f1INW0fJnHTFmbyF_0be5b4u-mozla
4f' -O steve-jobs-commencement.txt

loader = TextLoader("steve-Jobs-commencement.txt")
documents = loader.load()
text-splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                               chunk_overlap=0)
texts = text-splitter.split-documents(documents)
```

b) Step ② Embed the passages & Store them in an Index

This step is the same as in the 1st example, where we turn each chunk into a text embedding, using the same multilingual model, and index these embeddings in a vector database.

```
embeddings = CohereEmbeddings(model="multilingual-22-12")
db = Qdrant.from_documents(texts, embeddings,
                           location=":memory:",
                           collection_name="my_documents",
                           distance_func="Dot")
```

↳ Step ③: Enter a Question

Next, we enter a list of questions that we'd like to get answers for. Below are the five questions we'll ask. The first three are questions that require factual answers. The fourth is more of an opinion-based question with no one correct answer. Finally, the fifth question asks about factual info^n we know is unavailable in the context.

```
questions = [
    "What did the author liken the Whole Earth Catalog to ?",
    "What was Reed College great at ?",
    "What was the author diagnosed with ?",
    "What is the key lesson from this article",
    "What did the article say about Michael Jackson ?"
]
```

Let's see if we can get the answers we're expecting.

↳ Step ④: Answer the Question Based on the Most Relevant Passages

For each question, we need to find the chunks of text most relevant to the question. Here, we want to get the top-four most relevant chunks (a default number you can change)

We'll use these chunks as the context to answer the question. The bigger the context, the more info^n becomes available to the text generation model, which we'll use 'command' to generate the answer.

generate the
a prompt to
- An instructio
- The context
the most
- The ques

with langchain,
pack the whi
and calling

The method c
modify it usi
modifying th
returns bet

prompt
Questi
Answe
If th
that
PROM

cha

generate the answer, we need to provide the following a prompt to the text generation model.

- An instruction of what we want it to generate
- The context, which is the chunks of text that are the most relevant to the question
- The question.

With langchain, we can use the load_qa_chain method to pack the whole thing together: creating the prompt and calling the text generation model.

The method comes with a predefined prompt, but we can modify it using the PromptTemplate module. I found that modifying the prompt for this particular example returns better results & here's what it looks like:

```
prompt_template = """Text: {context}

Question: {question}
Answer the question based on the text provided.
If the text does not contain the answer, reply
that the answer is not available."""

PROMPT = PromptTemplate(
        template=prompt_template,
        input_variables=["context",
                         "question"]
        )

chain_type_kwargs={"prompt": PROMPT}
```

And here's what the code looks like:

```
qa = RetrievalQA.from_chain_type(
        llm = Cohere(model = "command",
            temperature = 0),
        chain_type = "stuff",
        retriever = db.as_retriever(),
        chain_type_kwargs = chain_type_kwargs,
        return_source_documents = True)

for question in questions:
    answer = qa({"query": question})
```

And below are the answers we get. They seem to be spot on!

The first three are factual answers, which the system extracts from the text. The fourth is an open-ended question, in which the system provides a reasonable answer based on passages in the text. And the fifth question is answered correctly, where there is no info^n from the text to answer the question.

Question: What did the author liken the Whole Earth Catalog to?
Answer: It was sort of like Google in paperback form . . . .

Question: What was Reed College great at?
Answer: Reed College was great at . . .

Question:
Answer:

Question:
Answer:

Question:
Answer

↳ we can see
in Langch

⇒ Final

In this artic

The first is th
model to eff
multilingual
need to buil
different
to cover

The second
app^n's wh
Simplifie
its m

Question: What was the author diagnosed with?
Answer: The author was diagnosed with cancer.

Question: what is the key lesson from this article?
Answer: The key lesson from this article is that . . .

Question: What did the article say about Michael Jackson
Answer: The text did not provide . . .

→ we can see these prompts by enabling the debug mode in Langchain like so:

```
impent langchain
langchain.debug = False
```

⇒ Final Thoughts.

In this article, we looked at two key things.
The first is the power of using a single multilingual embedding model to efficiently build a search application that requires multilingual support, a common need these days. There is no need to build complex interfaces & pipelines to support different langs. Instead, we can use one pipeline to cover lots languages.

The second is the ease & efficiency of building such app's when paired with the Langchain library. This simplifies the building of end-to-end pipelines via its modules such as Chains, Indexes & more

This is quite a powerful combination. It means you can build & tinker with multilingual semantic search applications (and beyond) with the least friction. The more efficient you are, the more experiments you can make & the faster you get to that app that truly makes a difference.

Use Case Patterns.

We'll go through several broad use case categories for the Command model.

↳ Cohere's Command model is an instruction-following text generation model trained to follow user commands. It is trained to be instantly useful in practical applications, covering a wide range of use cases.

↳ LLMs like the command model are general-purpose and can be applied in infinite ways, but if one can't recognize the patterns where they can be useful, it can feel overwhelming.

↳ In this chapter, we'll go through several broad use case categories for the Command model. Though they won't cover all the possible ways that you can use the model, they are good starting points for understanding the patterns of tasks where the model works well.

We'll go through the following use cases:

1. Writing
2. Question Answering
3. Brainstorming
4. Transforming
5. Summarizing
6. Rewriting
7. Extracting
8. Classifying.

→ Setting Up

For each use case, can be constructed

```
! pip install
import co
co = cohe
```

Let's also define temperature val which is hav We set a d nudges the less random response ge

```
def ge
res
```

for

Our examples for launchin getting the and so on.

→ Setting Up

In each use case, we'll look at some ideas on how a prompt can be constructed & the associated model settings.

```
! pip install cohere
import cohere
co = cohere.ClientV2("Cohere API key")
```

Let's also define a function to take a prompt & a temperature value & then call the chat endpoint, which is how we can access the command model. We set a default temperature value of 0, which nudges the response to be more predictable and less random. This function returns the text response generated by the model.

```
def generate_text(prompt, temp=0):
    response = co.chat_stream(
        message = prompt,
        model = "command-r-plus-08-2024",
        temperature = temp,
        preamble = " ")

    for event in response:
        if event.event_type == "text-generation":
            print(event.text, end=' ')
```

Our examples will revolve around a company's activities for launching a new wireless headphone product, such as getting the word out, managing customer interactions, and so on.

For this, let's define a text snippet containing the product description. We'll be utilizing the snippet in several examples throughout this article.

product = """ The CO-1T is a wireless headphone product that uses Bluetooth to connect to your devices. It has a long battery life & can be quickly charged using the - - — """

Writing.

We'll start with the most general type of use case, which is writing.

Here we can ask the model to write freeform text, for example, with this prompt: "Create an email about the launch of the wireless headphone product". But that alone might not be very useful in practical app's because the generated text can go in very different directions. We may want to add more context & specificity to the prompt.

Here's how we can do that. Let's say we're building an app for users to enter some bullet points & get a complete email written. We can set up the prompt in the following way: Create a variable for the user to input some text & merge that, together with the product description, into the main prompt.

```
user_input = """
    - announce product launch
    - create a call to action
    - mention live chat for support
"""
prompt = f """ {product}
Create an email about the product above
mentioning the following:
{user_input}
"""
response = generate_text (prompt, temp = 0.5)
print (response)
```

Adding this context helps to guide the model in the direction we want it to go. Here is a sample response:

"We are thrilled to announce the launch of our newest ---

In this example, we changed the temperature value to 0.5. And throughout this article, you'll see different temperature values being used in diff situations. Increasing the temperature value tells the model to generate less predictable responses & instead be more "creative". And since this task does require some form of creativity (expanding brief keywords into a full passage),

We increased the temperature value. There is no one right value for a use case, so this is a setting you should try & iterate to get the best outcome for your task.

We have looked at one example, but writing as a use case can mean many different things. So instead of beginning your prompt with "Create," you can try other ways to extract diff. forms of content & formats that your appn needs, such as using "Explain," "Generate", "Write," "Provide," "Design," "Describe," & so on.

↳ Question Answering.